

1. Judul Praktikum

Praktikum Struktur Data - Modul 4: Array dan List.

2. Tujuan Praktikum

1. Mengenal dan memahami konsep struktur data list dan array.
2. Mampu melakukan eksekusi dasar seperti menambah (insert), menghapus (delete), mengubah (update), dan membaca (traversal) isi list.
3. Menerapkan logika dasar untuk mencari data (searching) dan mengurutkan data (sorting).

3. Dasar Teori

List adalah tipe struktur data linear yang berfungsi menampung berbagai nilai ke dalam satu nama variabel. Di dalam bahasa Python, list sangat dinamis dan elemen di dalamnya dapat diubah kapan saja. Setiap elemen dalam list memiliki nomor urut yang disebut indeks, di mana elemen pertama selalu dimulai dengan indeks 0. Operasi pengurutan data dapat dilakukan menggunakan fungsi bawaan, sedangkan pencarian dilakukan dengan mencocokkan nilai satu per satu dari awal hingga akhir.

4. Langkah Praktikum

Berikut adalah baris kode untuk menjalankan prosedur praktikum dasar list serta penyelesaian untuk keempat studi kasus pengolahan data mahasiswa dan barang.

```
1 print("--- PROSEDUR PRAKTIKUM DASAR ---")
2
3 print("\n1. Membuat dan Menampilkan List")
4 data_list = [10, 20, 30, 40]
5 print("Data saat ini:", data_list)
6
7 print("\n2. Menambah dan Menghapus Data")
8 data_list.append(50)
9 print("Sesudah ditambah 50:", data_list)
10 data_list.remove(20)
11 print("Sesudah dihapus 20:", data_list)
12
13 print("\n3. Update Data")
14 data_list[1] = 100
15 print("Sesudah indeks 1 diubah jadi 100:", data_list)
16
17 print("\n4. Traversal Data")
18 for isi in data_list:
19     print(isi, end=" | ")
20 print()
21
22 print("\n5. Searching Data")
23 cari_angka = 100
24 for i in data_list:
25     if i == cari_angka:
26         print(f"Angka {cari_angka} berhasil ditemukan!")
27
28 print("\n6. Sorting Data")
29 acak = [40, 10, 30, 20]
30 acak.sort()
31 print("Data setelah diurutkan:", acak)
32
33
34 print("\n\n--- STUDI KASUS ---")
35
36 print("\nKasus 1: Data Nilai Mahasiswa")
37 kumpulan_nilai = [85, 90, 78, 92, 88]
38 print("Nilai seluruh mahasiswa:", kumpulan_nilai)
39
40 print("\nKasus 2: Pencarian Data Barang")
41 rak_barang = ["Buku", "Pensil", "Penghapus", "Penggaris"]
42 target_barang = "Pensil"
43 status_ketemu = False
44 for b in rak_barang:
45     if b == target_barang:
46         status_ketemu = True
47         break
48 print(f"Status pencarian '{target_barang}': {'Ditemukan' if status_ketemu else 'Kosong'}")
49
50 print("\nKasus 3: Pengurutan Nilai")
51 nilai_berantakan = [75, 60, 90, 50, 85]
52 nilai_berantakan.sort()
53 print("Nilai dari terkecil ke terbesar:", nilai_berantakan)
54
```

```
55 print("\nKasus 4: Update Data Mahasiswa")
56 nama_mhs = ["Andi", "Budi", "Citra"]
57 print("Data asli:", nama_mhs)
58 nama_mhs[1] = "Bambang"
59 print("Data setelah diperbarui:", nama_mhs)
```

--- PROSEDUR PRAKTIKUM DASAR ---

1. Membuat dan Menampilkan List

Data saat ini: [10, 20, 30, 40]

2. Menambah dan Menghapus Data

Sesudah ditambah 50: [10, 20, 30, 40, 50]

Sesudah dihapus 20: [10, 30, 40, 50]

3. Update Data

Sesudah indeks 1 diubah jadi 100: [10, 100, 40, 50]

4. Traversal Data

10 | 100 | 40 | 50 |

5. Searching Data

Angka 100 berhasil ditemukan!

6. Sorting Data

Data setelah diurutkan: [10, 20, 30, 40]

--- STUDI KASUS ---

Kasus 1: Data Nilai Mahasiswa

Nilai seluruh mahasiswa: [85, 90, 78, 92, 88]

Kasus 2: Pencarian Data Barang

Status pencarian 'Pensil': Ditemukan

Kasus 3: Pengurutan Nilai

Nilai dari terkecil ke terbesar: [50, 60, 75, 85, 90]

Kasus 4: Update Data Mahasiswa

Data asli: ['Andi', 'Budi', 'Citra']

Data setelah diperbarui: ['Andi', 'Bambang', 'Citra']

5. Analisis

- Fungsi `append()` selalu memasukkan data baru ke posisi paling belakang dari sebuah list.
- Penghapusan dengan `remove()` bekerja dengan mencari nilai yang persis sama, bukan berdasarkan urutan indeksinya.
- Algoritma pencarian linear memeriksa satu per satu elemen. Menambahkan perintah `break` sangat disarankan agar perulangan langsung berhenti ketika data sudah ditemukan, sehingga menghemat memori.

5. Kesimpulan

Pemahaman mengenai array dan list di Python adalah kemampuan mutlak yang harus dimiliki. List mempermudah kita mengelola kumpulan data tanpa harus membuat banyak variabel terpisah. Dengan mengombinasikan perulangan dan kondisional, kita bisa membuat sistem pengelolaan data yang canggih dari struktur yang sederhana.